



Universidade de Brasília (UNB)
Departamento de Ciência da Computação (CIC)
CIC 0105 - ENGENHARIA DE SOFTWARE - Turma 01 - 2026/1
Professor: Fernando Antonio de Araujo Chacon de Albuquerque

Alunos - Matrículas:
Caetano Korilo Fleury de Amorim - 212006737
Luis Henrique Wiltgen de Toledo - 221003968
Pedro Rafael Faria Ferreira - 190094591
Vinicius Chaves - 211060764

Especificação de Requisitos Não Funcionais (System-Wide Requirements)

Kanban Web

1. Introdução

Este documento especifica os requisitos não funcionais (RNF) do sistema Kanban Web, estabelecendo as diretrizes técnicas e os atributos de qualidade que regem o comportamento e as restrições operacionais da aplicação. Em consonância com a disciplina de Engenharia de Software da Universidade de Brasília, o detalhamento desses requisitos adota como referência o modelo internacional de qualidade de produto de software ISO/IEC 25010 (System and Software Quality Models). Os requisitos não funcionais não descrevem as funcionalidades ativas do sistema, mas determinam as restrições sob as quais as funcionalidades devem operar, assegurando características como eficiência de desempenho, confiabilidade, segurança, manutenibilidade, usabilidade, portabilidade e compatibilidade.

O correto levantamento e a consequente validação desses atributos atenuam riscos de falhas estruturais durante picos de acesso concorrentes, comuns em contextos de avaliação acadêmica, e blindam a aplicação contra vulnerabilidades de segurança clássicas descritas na literatura técnica. Ao longo deste documento, cada requisito é decomposto em critérios mensuráveis, justificativas plausíveis de projeto e estratégias de verificação prática, permitindo que a equipe de engenharia valide sua conformidade de forma inequívoca ao término de cada ciclo de desenvolvimento.

2. Desempenho (Performance Efficiency)

A eficiência de desempenho é uma qualidade essencial para aplicações interativas baseadas em quadros de tarefas, uma vez que a latência no processamento de requisições degrada diretamente a experiência do usuário durante reuniões de acompanhamento de projetos ou revisões rápidas. No desenvolvimento de ferramentas que adotam arquiteturas sem pipelines de compilação complexos (no-build full-stack), a otimização dos tempos de resposta do servidor e do carregamento de recursos estáticos torna-se o principal pilar para garantir uma sensação de fluidez e reatividade.



RNF-PERF-01 - Tempo de Resposta da API

Todas as requisições síncronas processadas pelo servidor backend devem apresentar baixa latência de resposta sob condições normais de uso.

Atributo	Valor
Identificador	RNF-PERF-01
Categoria	Desempenho / Eficiência de Execução
Descrição	Todas as requisições à API REST devem retornar resposta em tempo adequado.
Critério	Pelo menos 95% das requisições devem obter resposta em 500ms; nenhuma requisição isolada pode ultrapassar o teto absoluto de 2000ms.
Justificativa	Garantir fluidez na experiência do usuário durante operações de CRUD de cartões, colunas e quadros.
Métrica	Tempo medido a partir do recebimento da requisição HTTP pelo servidor ASGI até o envio completo da resposta.
Prioridade	Alta

O processamento das requisições assíncronas no framework FastAPI é executado de forma não-bloqueante por meio de um loop de eventos baseado em ASGI (Uvicorn). Para assegurar o cumprimento do limite de 500 ms nas operações de persistência, as sessões de banco de dados SQLAlchemy devem utilizar um mecanismo eficiente de pooling de conexões. Como o banco de dados utilizado é o SQLite (RNF-PORT-02), o tempo de processamento local é minimizado pela ausência de latência de rede entre o servidor de aplicação e o banco, porém exige um controle estrito para evitar gargalos durante a concorrência de escrita.

RNF-PERF-02 - Tempo de Carregamento do Frontend

A renderização visual inicial da interface cliente deve ocorrer de maneira ágil, garantindo que o usuário possa visualizar a estrutura básica da página sem atrasos perceptíveis.

Atributo	Valor
----------	-------



Identificador	RNF-PERF-02
Categoria	Desempenho
Descrição	A interface web deve carregar completamente em tempo aceitável
Critério	First Contentful Paint (FCP) ≤ 2 s em conexão de 10 Mbps
Justificativa	Primeira impressão do usuário determina a adoção da ferramenta
Prioridade	Média

A escolha arquitetural de não utilizar frameworks pesados do ecossistema Node.js (como React, Angular ou Vue) elimina o tempo de download de grandes pacotes de scripts JavaScript e processos dispendiosos de hidratação no cliente. A utilização de Alpine.js e Tailwind CSS servidos localmente simplifica a árvore do DOM e otimiza o carregamento dos assets. O carregamento do Tailwind CSS pode ser feito de forma enxuta via compilação utilitária das classes empregadas, mantendo o arquivo final de estilos abaixo de 100 kB e acelerando o índice de First Contentful Paint.

RNF-PERF-03 - Capacidade de Usuários Simultâneos

A infraestrutura do sistema deve ser capaz de suportar acessos paralelos equivalentes ao contingente de usuários previsto para atividades integradas.

Atributo	Valor
Identificador	RNF-PERF-03
Categoria	Desempenho / Escalabilidade
Descrição	O sistema deve suportar múltiplos usuários acessando simultaneamente a aplicação.
Critério	Suportar até 50 usuários concorrentes sem degradação perceptível de desempenho.
Justificativa	Contexto acadêmico da UnB, com turmas de Engenharia de Software compostas por até aproximadamente 50 alunos acessando o sistema ao mesmo tempo.
Prioridade	Média



Para avaliar a capacidade do sistema sob carga concorrente de forma rigorosa, aplica-se a Lei de Little, que estabelece a relação fundamental entre o número médio de requisições no sistema (L), a taxa média de chegada de requisições (λ , representativo de throughput) e o tempo médio de processamento de cada requisição (W):

Considerando um cenário extremo de pico acadêmico onde 50 usuários concorrentes efetuam requisições contínuas e interativas ao mesmo tempo, mantemos $L = 50$ requisições ativas. Sob o critério estabelecido pelo RNF-PERF-01, o tempo médio ideal de processamento é de $W = 0.5$ s. Substituindo esses parâmetros na equação para deduzir o throughput mínimo necessário que o servidor Uvicorn deve sustentar:

Para atender a essa demanda sem sofrer degradação de desempenho em máquinas de baixa especificação comuns no laboratório acadêmico, o Uvicorn deve ser instanciado em modo multiprocesso (workers múltiplos) para distribuir a carga entre os núcleos de processamento do processador host, minimizando o impacto de operações bloqueantes.

3. Confiabilidade (Reliability)

A confiabilidade assegura que a aplicação Kanban Web mantenha seu funcionamento livre de inconsistências mesmo diante de falhas de hardware, rede ou entradas anômalas de dados. A integridade dos dados inseridos pelos estudantes é crucial, de modo que interrupções no fornecimento de energia ou travamentos do servidor não corrompa os quadros de acompanhamento de projetos.

RNF-REL-01 - Integridade Transacional

As operações que alteram o estado persistente do sistema devem ser executadas de modo a garantir a consistência total dos dados.

Atributo	Valor
Identificador	RNF-REL-01
Categoria	Confiabilidade
Descrição	Operações de escrita devem ser atômicas – concluídas por completo ou revertidas
Critério	Nenhuma operação deve deixar o banco em estado inconsistente em caso de falha
Implementação	Uso de transações SQLAlchemy com <code>`db.commit()`</code> / <code>`db.rollback()`</code>
Prioridade	Alta

O padrão arquitetural de injeção de dependência de sessão (`get_db`) assegura que cada requisição HTTP receba sua própria conexão isolada com o banco de dados relacional. O ciclo de vida da sessão é controlado de maneira que erros inesperados durante o



processamento de regras de negócio acionem o rollback automático da transação, mantendo as constraints do banco invioladas e anulando gravações parciais.

RNF-REL-02 - Disponibilidade

A aplicação deve permanecer acessível nos horários de maior demanda e realização de atividades acadêmicas.

Atributo	Valor
Identificador	RNF-REL-02
Categoria	Confiabilidade
Descrição	O sistema deve estar disponível durante os períodos de uso ativo
Critério	Disponibilidade $\geq$ 99% durante horário comercial (08h-22h, dias úteis)
Justificativa	Uso acadêmico com prazos de entrega
Prioridade	Média

Considerando a janela de tempo comercial de uso ativo especificada no critério (14 horas por dia, de segunda a sexta-feira, totalizando aproximadamente 280\text{ horas} de operação mensal planejada), a taxa de disponibilidade de 99\% restringe o tempo de indisponibilidade não programada a um teto máximo tolerável de apenas 2.8\text{ horas} por mês. Para mitigar indisponibilidades prolongadas, a execução do servidor FastAPI deve ser gerenciada localmente por um monitor de processos ou serviço de sistema operacional (como systemd no Linux) configurado para reiniciar a aplicação de forma autônoma perante falhas inesperadas.

RNF-REL-03 - Tratamento de Erros

O comportamento do sistema frente a exceções deve ser padronizado de forma a não expor dados internos ou comprometer a segurança da aplicação.

Atributo	Valor
Identificador	RNF-REL-03
Categoria	Confiabilidade
Descrição	O sistema deve retornar mensagens de erro claras e não expor stack traces
Critério	Respostas de erro sempre contêm campo <code>`detail`</code> com mensagem descritiva em português; código HTTP correto
Implementação	<code>`HTTPException`</code> do FastAPI com mensagens padronizadas
Prioridade	Alta

O vazamento de informações técnicas por meio de depurações expostas (stack traces brutas do Python) facilita a identificação de vulnerabilidades estruturais por agentes maliciosos. O uso de blocos try-except robustos em nível de roteadores captura os erros operacionais do SQLAlchemy e os converte em respostas limpas com códigos HTTP



semânticos, mascarando o esquema interno das tabelas de banco de dados.

4. Segurança (Security)

A segurança em sistemas de gerenciamento de tarefas compartilhados envolve a garantia de privacidade das informações dos projetos acadêmicos e a proteção das contas dos alunos contra acessos não autorizados. A implementação de controles estritos no backend, independente de validações auxiliares executadas na interface do usuário, assegura a integridade das operações.

RNF-SEC-01 - Validação de Entrada no Backend

Todas as informações recebidas pelo servidor de aplicação devem ser submetidas a uma validação estrita antes do seu processamento.

Atributo	Valor
Identificador	RNF-SEC-01
Categoria	Segurança / Proteção de Dados
Descrição	Toda entrada de dados deve ser validada e sanitizada no backend, independentemente do frontend.
Critério	Definição de schemas Pydantic com regras de validação aplicadas em todos os endpoints de escrita da API.
Justificativa	Prevenção ativa contra vetores de injeção de dados e scripts maliciosos (alinhado a OWASP A03:2021 - Injection).
Prioridade	Alta

A validação de payloads de requisição no FastAPI ocorre de maneira automática através da interpretação de classes baseadas em Pydantic. Caso os parâmetros enviados não correspondam estritamente aos tipos e regras definidos no backend (por exemplo, strings contendo caracteres de controle ou tamanhos excedentes ao limite), o sistema rejeita a requisição imediatamente retornando um código 422 Unprocessable Entity, impedindo que os dados nocivos alcancem as instruções de consulta ao banco de dados.

RNF-SEC-02 - Controle de Autorização por Recurso

O sistema deve certificar-se de que cada operação em um recurso específico seja efetuada por um usuário devidamente autenticado e autorizado.

Atributo	Valor
Identificador	RNF-SEC-02



Atributo	Valor
Categoria	Segurança / Controle de Acesso
Descrição	Operações destrutivas ou de gerenciamento em quadros só podem ser realizadas por seus respectivos donos.
Critério	Endpoints de exclusão de quadro e gestão de membros comparam o identificador do usuário logado com o campo email_dono do quadro, retornando código 403 Forbidden perante divergências.
Justificativa	Prevenir a usurpação de privilégios e alteração não autorizada de dados de terceiros (OWASP A01:2021 - Broken Access Control).
Prioridade	Alta

O controle de privilégios é exercido de forma imperativa na camada de roteadores do FastAPI. Toda rota sensível recebe o identificador do usuário extraído e validado por meio de dependências de segurança (tokens JWT ou cookies de sessão). O sistema executa uma consulta prévia para obter as permissões do quadro requisitado antes de aplicar qualquer alteração lógica, mitigando riscos de acesso direto a objetos vulneráveis (IDOR).

RNF-SEC-03 - Unicidade de Recursos

Garantir a integridade referencial dos dados impedindo a existência de duplicidades de registros cruciais para a lógica de negócios da aplicação.

Atributo	Valor
Identificador	RNF-SEC-03
Categoria	Segurança / Integridade
Descrição	O sistema deve garantir unicidade de e-mail de conta e código de quadro
Critério	Tentativas de duplicação retornam 409 Conflict com mensagem explicativa
Implementação	Chaves primárias e constraints UniqueConstraint no banco de dados



Prioridade	Alta
------------	------

A consistência de registros únicos não depende meramente da validação lógica das camadas superiores da aplicação, sendo assegurada pelas regras impostas diretamente no mecanismo de banco de dados SQLite. Isso previne falhas causadas por condições de corrida (race conditions) quando requisições concorrentes idênticas tentam persistir o mesmo e-mail simultaneamente.

RNF-SEC-04 - CORS Configurado

Restringir a interação de outros sistemas e scripts externos com a API REST exposta pelo servidor.

Atributo	Valor
Identificador	RNF-SEC-04
Categoria	Segurança / Proteção de Rede
Descrição	O middleware de Cross-Origin Resource Sharing (CORS) deve estar ativado e configurado de forma restritiva no backend.
Critério	CORS habilitado via CORSMiddleware no FastAPI.
Observação	Em ambiente de produção, o parâmetro allow_origins deve referenciar exclusivamente o domínio de hospedagem do frontend do sistema.
Prioridade	Média

Ao isolar a API de requisições maliciosas disparadas a partir de outras abas ou sites maliciosos nos navegadores dos usuários, o middleware CORS atua como uma barreira preventiva contra o roubo de dados de sessões acadêmicas ativas.

RNF-SEC-05 - Validação de Regras de Senha

Impor padrões mínimos de complexidade estrutural nas senhas cadastradas para aumentar a resistência das credenciais contra ataques de brute-force.

Atributo	Valor
Identificador	RNF-SEC-05
Categoria	Segurança / Autenticação Forte
Descrição	Senhas de usuários devem obedecer a regras estritas de complexidade mínima.
Critério	Exatamente 5 caracteres únicos contendo obrigatoriamente: \ge 1 letra



Atributo	Valor
	maiuscula, ≥ 1 letra minúscula, ≥ 1 dígito numérico e ≥ 1 caractere especial de pontuação.
Justificativa	Ampliar a entropia e a robustez criptográfica da senha mesmo sob a restrição acadêmica de comprimento reduzido.
Prioridade	Alta

Análise Matemática de Entropia e Complexidade de Senhas

O limite rígido de comprimento estabelecido pelo critério acadêmico de desenvolvimento do projeto impõe que a senha possua exatamente 5 caracteres. Para elevar a resistência do sistema contra ataques de adivinhação sem poder expandir o tamanho do segredo, a restrição proíbe repetições e força a presença de caracteres pertencentes a quatro classes bem definidas de caracteres ASCII imprimíveis :

1. Letras maiúsculas (U): de 'A' a 'Z', onde $|U| = 26$.
2. Letras minúsculas (L): de 'a' a 'z', onde $|L| = 26$.
3. Dígitos numéricos (D): de '0' a '9', onde $|D| = 10$.
4. Caracteres de pontuação e especiais (P): de tamanho $|P| = 32$.

Como a senha tem tamanho 5 e deve conter ao menos um caractere de cada uma das quatro categorias exclusivas, a distribuição de caracteres na senha obedecerá, pelo Princípio da Casa dos Pombos, a uma partição de tipo $\{2, 1, 1, 1\}$. Isso significa que exatamente uma das categorias fornecerá dois caracteres à senha, enquanto as outras três categorias fornecerão exatamente um caractere cada. Como a senha veta repetições, os dois caracteres da categoria duplicada serão obrigatoriamente distintos.

O número de maneiras de selecionar as quantidades de caracteres para cada uma das quatro categorias de acordo com essa distribuição corresponde às permutações lógicas do conjunto $\{2, 1, 1, 1\}$, resultando em exatamente 4 cenários excludentes:

- Cenário 1: Categoria U fornece 2 caracteres; as demais fornecem 1 caractere cada.
- Cenário 2: Categoria L fornece 2 caracteres; as demais fornecem 1 caractere cada.
- Cenário 3: Categoria D fornece 2 caracteres; as demais fornecem 1 caractere cada.
- Cenário 4: Categoria P fornece 2 caracteres; as demais fornecem 1 caractere cada.

Calcula-se a quantidade de maneiras de escolher caracteres distintos para cada grupo



combinando-os de forma simples. O número total de combinações não ordenadas de caracteres selecionados para compor a senha é dado pela soma das possibilidades de escolha de cada cenário:

Efetando o cálculo dos coeficientes binomiais:

Substituindo os valores obtidos:

Para cada uma das combinações compostas por 5 caracteres distintos e únicos obtidos, as permutações internas que definem sua disposição linear na string final de senha são dadas por $5! = 120$. Logo, o número total de senhas válidas estruturalmente possíveis (N_{valid}) equivale a:

Admitindo uma distribuição de probabilidade perfeitamente uniforme no espaço de senhas válidas, a entropia de Shannon de informação contida na senha (H) pode ser calculada como:

Essa entropia de aproximadamente 30 bits posiciona a robustez da credencial em um nível satisfatório para sistemas web acadêmicos locais, demandando recursos computacionais e tentativas de invasão inviáveis de serem processadas em tempo real sob os limites de requisições concorrentes e bloqueios lógicos definidos para a aplicação.

5. Manutenibilidade (Maintainability)

A manutenibilidade define o esforço e os custos associados à introdução de modificações corretivas, evolutivas ou de adaptação ao longo do ciclo de vida da aplicação. A adoção de boas práticas de modularização e separação de conceitos facilita a integração de novos colaboradores e o desenvolvimento concorrente de equipes nas disciplinas de Engenharia de Software da UnB.

RNF-MANT-01 - Separação de Responsabilidades (Camadas)

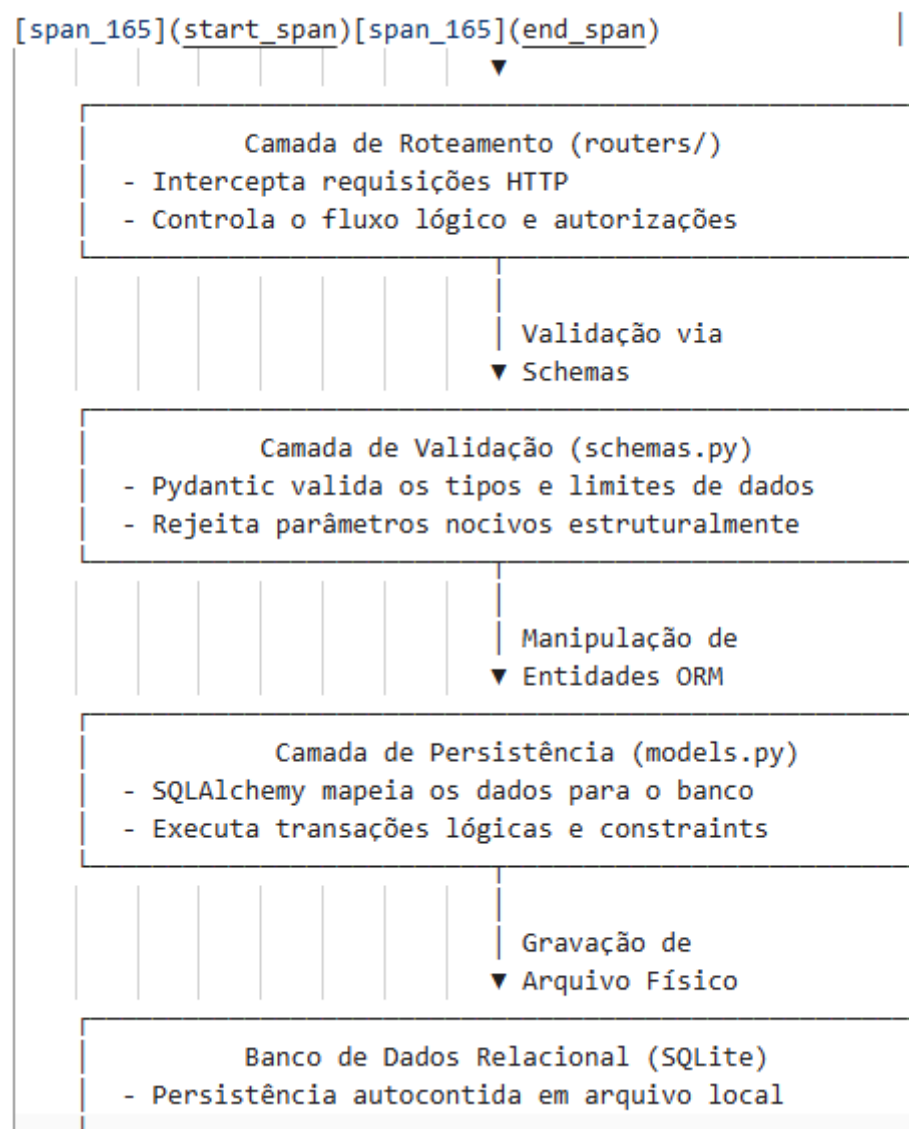
A arquitetura de código do sistema deve obedecer a uma distribuição lógica e física bem definida, prevenindo o acoplamento excessivo de componentes.

Atributo	Valor
Identificador	RNF-MANT-01
Categoria	Manutenibilidade / Arquitetura de Software
Descrição	O código do sistema deve ser estruturado em camadas lógicas com responsabilidades bem isoladas e delimitadas.
Critério	Separação em: <code>models.py</code> (mapeamento ORM) , <code>schemas.py</code> (validação e



Atributo	Valor
	serialização de dados) , routers/ (camada de controle e controle de rotas de API) e database.py (gestão de conexões e sessões de persistência).
Prioridade	Alta

Modelo de Arquitetura em Camadas (Fluxo de Dados)



A segregação do sistema em módulos específicos impede que modificações estruturais na persistência exijam correções em massa na interface com o usuário ou na lógica de controle. Essa arquitetura assegura que as regras de persistência residam exclusivamente na camada de dados, enquanto as checagens operacionais sejam confinadas aos roteadores.

RNF-MANT-02 - Nomenclatura em Português

A padronização linguística do código do projeto deve ser preservada em consonância com o domínio acadêmico estabelecido na disciplina.



Atributo	Valor
Identificador	RNF-MANT-02
Categoria	Manutenibilidade
Descrição	Nomes de variáveis, constantes, funções, classes, tabelas e endpoints devem ser redigidos na língua portuguesa.
Critério	Consistência de vocabulário com o escopo do projeto pedagógico e com os módulos legados que realizam integrações sistêmicas.
Prioridade	Média

Evita-se a mistura confusa de termos em inglês e português (*portinglish*) nas declarações de classes e tabelas. Essa uniformidade agiliza as sessões de revisão técnica de código (*code review*) executadas pelos engenheiros e avaliadores do projeto.

RNF-MANT-03 - Routers Modulares

A estrutura de roteamento HTTP do backend deve ser segregada em blocos modulares autônomos.

Atributo	Valor
Identificador	RNF-MANT-03
Categoria	Manutenibilidade / Modularidade
Descrição	Cada domínio lógico de negócio da aplicação deve possuir seu próprio router isolado no FastAPI.
Critério	Separação física de arquivos e namespaces lógicos: auth.py, contas.py, quadros.p[span_6](start_span)[span_6](end_span)[span_11](start_span)[span_11](end_span)y, cartoes.py e membros.py.
Prioridade	Alta

A modularização por domínios de rotas em arquivos separados sob o diretório routers/ simplifica a árvore de desenvolvimento, de forma que conflitos de mesclagem de código (*merge conflicts*) em sistemas de controle de versão (como Git) sejam atenuados quando múltiplos engenheiros trabalham simultaneamente no projeto.

RNF-MANT-04 - Schemas Reutilizáveis

Garantir o reaproveitamento máximo de lógica de validação de dados evitando redundâncias estruturais no backend.



Atributo	Valor
Identificador	RNF-MANT-04
Categoria	Manutenibilidade / Reusabilidade
Descrição	Funções de validação de domínio semântico devem ser centralizadas e reutilizáveis entre schemas distintos.
Critério	Centralização das rotinas validar_codigo, validar_email e validar_senha no módulo schemas.py, sendo herdadas ou importadas pelas demais estruturas.
Prioridade	Alta

O princípio DRY (*Don't Repeat Yourself*) é formalizado por meio da reutilização de componentes Pydantic. Qualquer modificação na regra estrutural de formatação de e-mails ou no tamanho das senhas é efetuada em um único ponto isolado do arquivo schemas.py, repercutindo automaticamente em todas as validações ativas na API.

6. Usabilidade (Usability)

A usabilidade do Kanban Web garante que os usuários consigam gerenciar seus projetos de forma ágil e livre de barreiras de navegação. Por se tratar de um sistema altamente visual e focado na manipulação dinâmica de elementos (cartões e colunas), a suavidade das interações na interface gráfica determina o sucesso operacional do produto final.

RNF-USA-01 - Interface Responsiva

A interface visual da aplicação cliente deve adaptar-se de maneira flexível à resolução dos terminais de acesso utilizados pelos alunos.

Atributo	Valor
Identificador	RNF-USA-01
Categoria	Usabilidade / Adaptabilidade
Descrição	A interface gráfica deve se adaptar de forma fluida a múltiplos tamanhos de tela.
Critério	Operação livre de distorções visuais ou perdas de funcionalidade em resoluções de 375\text{ px} (mobile) a 1920\text{ px} (desktop).
Implementação	Uso de classes utilitárias de grid responsivo, flexbox e media queries integradas do Tailwind CSS.
Prioridade	Média

Nos smartphones e telas estreitas (375\text{ px}), a visualização de múltiplas colunas



paralelas de um quadro Kanban é reestruturada automaticamente, aplicando barras de rolagem horizontal suave no contêiner do quadro ou colapsando colunas secundárias para preservar a legibilidade e a usabilidade.

RNF-USA-02 - Feedback Visual Imediato

As ações executadas pelo usuário na tela devem receber confirmação imediata de estado pela interface gráfica.

Atributo	Valor
Identificador	RNF-USA-02
Categoria	Usabilidade / Operabilidade
Descrição	O sistema deve fornecer feedback interativo imediato para todas as ações iniciadas pelo usuário na interface.
<i>Critério</i>	Notificações do tipo toast de curta duração exibidas em tela para sucessos (verde) e erros (vermelho) após cada transação.
Prioridade	Alta

Por meio da reatividade oferecida pelo Alpine.js, qualquer operação de envio de formulários ou reordenação de cartões dispara um toast visual sem interromper a tela ativa, assegurando que o usuário tome ciência do resultado da operação sem requerer atualizações completas de página.

RNF-USA-03 - Drag-and-Drop Intuitivo

O reposicionamento de cartões de tarefas e colunas deve ser executado de forma direta e tátil.

Atributo	Valor
Identificador	RNF-USA-03
Categoria	Usabilidade / Operabilidade
Descrição	Cartões de tarefas e colunas devem permitir a movimentação de status por meio de arrastamento visual fluido na tela.
<i>Critério</i>	Drag-and-drop funcional para mover cartões; alteração de cursor para <i>grabbing</i> ; realce com borda pontilhada ou sombreado translúcido no contêiner de destino potencial.
Implementação	Uso direto dos eventos da API nativa HTML5 Drag and Drop (dragstart, dragover, dragleave, drop).



Atributo	Valor
Prioridade	Alta

Mitigação de Efeitos de Cintilação (Flickering) em Elementos Filho

Um problema comum no desenvolvimento de painéis interativos utilizando a API de Drag-and-Drop nativa do HTML5 é a cintilação constante de classes de estilo de destaque (*flickering*) que ocorre quando o cursor de arraste passa por cima de elementos filhos contidos dentro da coluna de destino (como títulos, botões de ação ou parágrafos de texto).

Sob a especificação do HTML5, toda vez que o cursor paira sobre um nó filho de uma zona de drop, o navegador dispara um evento `dragleave` no contêiner pai e um evento `dragenter` no elemento filho. Se a reatividade do Alpine.js estiver configurada para remover as classes de destaque da coluna no evento `dragleave`, a interface piscará intermitentemente conforme o usuário move o cartão sobre os itens existentes na coluna.

Para solucionar este comportamento e assegurar a suavidade exigida pelo critério de aceitação, duas abordagens lógicas concorrentes são analisadas na tabela comparativa abaixo:

Parâmetro de Comparação	Abordagem 1: Máscara CSS	Abordagem 2: Contador de Entradas/Saídas de Eventos
Mecanismo Técnico	Aplicação temporária da propriedade <code>pointer-events: none</code> em todos os elementos descendentes das colunas e dos cartões enquanto o arraste está ativo.	Monitoramento de uma variável contadora inteira no estado reativo do Alpine.js. O valor é incrementado no <code>dragenter</code> e decrementado no <code>dragleave</code> .
Vantagens	- Solução declarativa de baixo custo de processamento; Dispensa a manutenção de estado JavaScript adicional para cada coluna de destino.	- Não altera o comportamento nativo de cliques dos elementos filhos; - Mantém a operabilidade de botões ou links durante o arraste caso necessário.
Desvantagens	- Exige a inserção de classes condicionais do Tailwind CSS em tempo de arraste	- Maior complexidade de código JavaScript para gerenciamento de listeners nos ciclos de vida ; -



Parâmetro de Comparação	Abordagem 1: Máscara CSS pointer-events: none	Abordagem 2: Contador de Entradas/Saídas de Eventos
	(pointer-events-none).	Bug potencial se o contador ficar dessincronizado devido a perdas de eventos.
<i>Conformidade Recomendada</i>	Alta recomendação: Ideal para manter a simplicidade arquitetural da aplicação Kanban Web, minimizando o volume de código JavaScript executado no cliente.	Média recomendação: Indicado apenas se houver necessidade estrita de manter elementos altamente interativos ativos durante o arrastar.

RNF-USA-04 - Validação com Mensagens de Erro Descritivas

A sinalização de erros na interface do usuário deve ser redigida de forma humanizada e livre de jargões técnicos crus.

Atributo	Valor
Identificador	RNF-USA-04
Categoria	Usabilidade / Tratamento de Erros
Descrição	Erros de validação semântica devem ser expostos ao usuário em linguagem natural clara e explicativa.
Critério	Mensagens de erro apresentadas no interior de modais de inserção ou como blocos de alerta sob os campos incorretos; vedação absoluta de exibição de stack traces brutas em tela.
Prioridade	Alta

As mensagens enviadas pelo backend em formato JSON através do campo detail (conforme especificado no RNF-REL-03) devem ser interceptadas pelo controlador em Alpine.js e expostas de forma amigável. Por exemplo, em caso de erro 409 Conflict no e-mail de cadastro, o sistema deve apresentar o texto: *"Este e-mail já está sendo utilizado por outra conta. Por favor, utilize outro endereço ou recupere seu acesso"*.

RNF-USA-05 - Compatibilidade com Navegadores

A interface de usuário da aplicação deve funcionar de forma estável nas versões modernas dos navegadores disponíveis no ambiente acadêmico.



Atributo	Valor
Identificador	RNF-USA-05
Categoria	Usabilidade / Portabilidade
Descrição	A interface web deve ser renderizada corretamente nos principais navegadores modernos sem perdas de layout ou scripts.
Critério	Compatibilidade assegurada com Chrome 110+, Firefox 110+, Edge 110+ e Safari 16+.
Prioridade	Média

A conformidade com as diretrizes do consórcio W3C na escrita de elementos HTML5 estruturais e o uso de recursos padrões de JavaScript (ES6) reduzem o risco de comportamentos discrepantes ou mal funcionamento da reatividade nos navegadores homologados.

7. Portabilidade (Portability)

A portabilidade determina a flexibilidade do sistema para rodar em diferentes ambientes operacionais sem exigir refatorações estruturais do backend. No contexto da UnB, os estudantes utilizam computadores de laboratório com distribuições Linux, enquanto muitos desenvolvem localmente em sistemas Windows ou macOS, tornando a portabilidade essencial.

RNF-PORT-01 - Independência de Sistema Operacional

A infraestrutura lógica da aplicação backend deve poder ser executada em múltiplas plataformas de sistemas operacionais.

Atributo	Valor
Identificador	RNF-PORT-01
Categoria	Portabilidade
Descrição	O backend da aplicação deve rodar nativamente sob Windows, distribuições Linux e macOS.
Critério	Instalação e execução documentadas para Windows (PowerShell), Linux/macOS e ambiente de compatibilidade WSL.
Implementação	Empregar estritamente dependências interpretadas Python e persistência local SQLite, sem chamadas de sistema específicas de SO.
Prioridade	Alta



O uso exclusivo da biblioteca padrão do Python e de dependências portáteis instaláveis via pip assegura que o servidor de aplicação se comporte de forma idêntica em qualquer plataforma. A manipulação de arquivos físicos e caminhos de diretórios deve utilizar de forma estrita a biblioteca nativa pathlib do Python, evitando erros comuns associados à diferença de separadores de diretório entre Windows (\) e Linux (/).

RNF-PORT-02 - Banco de Dados Autocontido

A persistência de dados não deve exigir o provisionamento de servidores de banco de dados complexos ou dependências externas em rede.

Atributo	Valor
Identificador	RNF-PORT-02
Categoria	Portabilidade / Facilidade de Instalação
Descrição	O banco de dados relacional deve ser autocontido e inicializado automaticamente pelo backend no primeiro ciclo de execução.
Critério	Criação automática do arquivo <code>kanban.db</code> no boot do sistema por meio da chamada de metadados <code>Base.metadata.create_all</code> do ORM.
Prioridade	Alta

A adoção do SQLite elimina a necessidade de instalação e configuração prévia de instâncias pesadas de SGBDs (como PostgreSQL, MySQL ou Oracle), o que simplifica drasticamente a implantação local por novos desenvolvedores.

8. Compatibilidade (Compatibility)

Compatibilidade representa a capacidade do Kanban Web de coexistir e integrar-se com outros produtos e padrões tecnológicos estabelecidos no ecossistema de software.

RNF-COMP-01 - API REST Padrão HTTP

A interface de programação exposta pelo backend deve seguir os conceitos de design consagrados para serviços baseados em rede.

Atributo	Valor
Identificador	RNF-COMP-01
Categoria	Compatibilidade / Interoperabilidade
Descrição	A API backend deve ser desenvolvida de acordo com os princípios arquiteturais REST, utilizando corretamente os métodos HTTP.
Critério	Uso correto de: GET para leitura; POST para inserções; PUT/PATCH para



Atributo	Valor
	edições; DELETE para exclusões de entidades; e códigos de retorno semanticamente coerentes.
Prioridade	Alta

O cumprimento estrito do protocolo HTTP permite que qualquer outro sistema terceiro, cliente móvel ou ferramenta de testes integrados (como Postman ou curl) consiga interagir com o Kanban Web de forma previsível e normatizada.

RNF-COMP-02 - Documentação OpenAPI

A documentação das capacidades da API deve ser disponibilizada de forma legível para máquinas e desenvolvedores.

Atributo	Valor
Identificador	RNF-COMP-02
Categoria	Compatibilidade / Documentabilidade
Descrição	A API REST exposta pelo servidor deve ser documentada de forma automatizada sob a especificação OpenAPI 3.0.
Critério	Swagger UI acessível e interativo no endereço /docs e ReDoc acessível na rota /redoc.
Implementação	Geração automática baseada em tipos estáticos de rotas e schemas pelo framework FastAPI.
Prioridade	Média

A auto-documentação em tempo de execução acelera as atividades de validação e o desenvolvimento paralelo do frontend, permitindo testes interativos diretos no navegador sem a necessidade de manter planilhas ou documentos manuais desatualizados de rotas.

9. Restrições de Implementação

As restrições de implementação definem os limites do espaço de solução de projeto de engenharia, impostos de forma imperativa por decisões administrativas, pedagógicas ou limitações do próprio ambiente onde o sistema será operado.

RNF-IMPL-01 - Stack Tecnológica Obrigatória

O ecossistema técnico de linguagens e ferramentas para o desenvolvimento do sistema é fixado pelas diretrizes curriculares do projeto acadêmico.



Atributo	Valor
Identificador	RNF-IMPL-01
Categoria	Restrição Tecnológica
Descrição	O sistema deve ser desenvolvido fazendo uso estrito das tecnologias estipuladas no escopo do projeto acadêmico.
Critério	Backend: Python 3.10+ com FastAPI e SQLAlchemy ; Frontend: Alpine.js e Tailwind CSS ; Banco de Dados: SQLite.
Prioridade	Alta (Restrição Acadêmica)

O cumprimento rígido dessa restrição é monitorado ao longo das avaliações do projeto acadêmico, sendo proibida a substituição unilateral de qualquer elemento da pilha de desenvolvimento. A combinação de ferramentas selecionadas oferece alta reatividade na interface web e simplicidade na persistência local.

RNF-IMPL-02 - Sem Dependências de Serviços Externos em Runtime

O sistema deve possuir autonomia operacional total, mantendo-se plenamente funcional em condições de desconexão externa.

Atributo	Valor
Identificador	RNF-IMPL-02
Categoria	Restrição de Execução
Descrição	O funcionamento da aplicação em ambiente local não deve depender de conexão com serviços de terceiros baseados na nuvem.
Critério	Dependências de terceiros declaradas em requirements.txt ; biblioteca Alpine.js servida diretamente a partir do diretório local /static/ do backend.
Observação	Embora a CDN do Tailwind CSS seja aceitável em ambiente de desenvolvimento inicial, para implantação de produção recomenda-se a compilação local de estilos.
Prioridade	Média

Ao isolar a aplicação de fatores externos (como indisponibilidades em CDNs ou APIs de serviços terceiros na internet), a restrição assegura que o Kanban Web se mantenha operacional durante a realização de laboratórios práticos presenciais na universidade.

10. Rastreabilidade

A rastreabilidade dos requisitos não funcionais mapeia de forma transparente a relação



direta entre as qualidades abstratas especificadas neste documento e os componentes físicos de código e infraestrutura responsáveis por sua consolidação material no produto de software. Essa rastreabilidade viabiliza auditorias técnicas estruturadas e garante que os critérios de qualidade sejam mantidos ao longo do processo de evolução sistêmica.

Identificador RNF	Componente de Implementação Concreta	Estratégia de Verificação e Testes
RNF-PERF-01	Processamento assíncrono nativo no ecossistema ASGI (uvicorn(async) associado ao pooling de conexões do SQLAlchemy (QueuePool).	Execução de perfilamento de rotas (<i>profiling</i>) e monitoramento de tempos de resposta via middleware de telemetria HTTP.
RNF-PERF-02	Estruturação de interface leve baseada em Alpine.js e folha de estilos Tailwind CSS compactada e servida localmente.	Medição de tempo de renderização utilizando a ferramenta de auditoria de navegadores Lighthouse.
RNF-PERF-03	Configuração de multiprocessamento de instâncias do Uvicorn no host associado a ajustes de timeout no SQLite.	Testes de injeção de carga simulada com ferramentas de estresse de requisições (como Locust ou Apache Benchmark).
RNF-REL-01	Implementação de blocos try/except nas camadas do controlador gerenciando commits e rollbacks estruturados (db.commit() / db.rollback()).	Testes unitários de transações de banco simulando falhas abruptas para verificar rollback automático.
RNF-REL-02	Inicialização do servidor configurada sob um monitor de processos do sistema operacional (como systemd ou pm2) para auto-recuperação.	Análise de relatórios mensais de uptime gerados por scripts de ping operantes na infraestrutura de testes.
RNF-REL-03	Emprego sistemático de exceções de classe	Testes de integração enviando payloads inválidos e verificando o



Identificador RNF	Componente de Implementação Concreta	Estratégia de Verificação e Testes
	HTTPException do FastAPI capturando falhas sem vazarem stack traces.	encapsulamento de segurança da resposta de erro JSON.
RNF-SEC-01	Definição de regras de validação declarativa utilizando schemas baseados em Pydantic no arquivo schemas.py.	Testes de carga enviando payloads intencionalmente malformados e strings excedentes para checar rejeição ativa.
RNF-SEC-02	Implementação de dependências de checagem em nível de rota comparando o ID autenticado com o atributo email_dono do recurso.	Testes funcionais com usuários distintos simulando acessos a identificadores de quadros pertencentes a terceiros.
RNF-SEC-03	Definições de chaves primárias e constraints estruturais UniqueConstraint aplicadas sobre as tabelas lógicas em models.py.	Teste de persistência concorrente tentando cadastrar e-mails ou códigos de quadros duplicados no banco.
RNF-SEC-04	Registro do middleware nativo CORSMiddleware parametrizado na inicialização da aplicação FastAPI.	Testes automatizados enviando requisições do tipo Preflight (OPTIONS) simulando origens web externas não homologadas.
RNF-SEC-05	Validador do tipo @field_validator incorporado no schema de criação de conta no arquivo schemas.py	Testes unitários disparando senhas de comprimentos distintos ou desprovidas das classes obrigatórias de complexidade.



Identificador RNF	Componente de Implementação Concreta	Estratégia de Verificação e Testes
RNF-MANT-01	Organização física de diretórios e arquivos em camadas: models.py, schemas.py, database.py e subpastas routers/.	Revisão estática da estrutura do projeto de software visando homologar o isolamento das dependências.
RNF-MANT-02	Uso rigoroso de português na codificação de classes, endpoints, variáveis e tabelas nas rotinas de engenharia.	Inspeção visual de código e verificação assistida por linter customizado nas sessões de integração de código.
RNF-MANT-03	Modularização lítica de rotas instanciando objetos APIRouter individuais no diretório routers/.	Verificação da árvore física do projeto assegurando que nenhuma lógica de domínio se infiltre no arquivo main.py.
RNF-MANT-04	Centralização e reutilização de validadores semânticos de formatos no arquivo schemas.py do backend.	Verificação estática de duplicação de lógica de tratamento de dados entre schemas distintos de payloads.
RNF-USA-01	Grid fluido estruturado por classes responsivas nativas do Tailwind CSS no arquivo static/index.html.	Testes visuais manuais e automatizados utilizando emuladores de múltiplos viewports de dispositivos móveis.
RNF-USA-02	Integração de componente de exibição reativa toast sob controle do estado global do Alpine.js.	Testes funcionais simulando respostas de sucesso e erro e inspecionando o surgimento e autolimpeza das notificações.
RNF-USA-03	Mapeamento de escutas nativas DragEvents e isolamento visual dos filhos com CSS pointer-events[span_250](start_span)[span_250](end_span)-none.	Testes de usabilidade interativos efetuando o arraste de cartões em direções variadas com o mouse.



Identificador RNF	Componente de Implementação Concreta	Estratégia de Verificação e Testes
RNF-USA-04	Tratamento de requisições HTTP malformadas no Alpine.js expondo o campo de mensagem detail em português em tela.	Geração intencional de falhas semânticas em formulários avaliando a clareza e localização do erro retornado em tela.
RNF-USA-05	Emprego de bibliotecas portáteis JavaScript e compilação de CSS em conformidade com os padrões modernos W3C.	Testes de aceitação manual verificando o comportamento da interface web em múltiplos motores de renderização.
RNF-PORT-01	Utilização exclusiva da biblioteca padrão do Python e dependências multiplataformas no requirements.txt.	Instalação, inicialização e execução automatizada do servidor de aplicação em hosts Linux, Windows e macOS.
RNF-PORT-02	Invocação do script utilitário de geração de tabelas create_all executado na inicialização lida em main.py.	Exclusão física do arquivo de banco kanban.db e verificação da sua recriação autônoma no novo boot de aplicação.
RNF-COMP-01	Implementação de rotas e decoradores específicos para cada verbo semântico correspondente na camada de controle.	Inspeção das requisições HTTP disparadas pela interface cliente por meio do console de inspeção de rede do navegador.
RNF-COMP-02	Configurações nativas automáticas sob o framework FastAPI gerando as páginas de documentação de rotas.	Acesso direto às rotas /docs e /redo [span_175](start_span)[span_175](end_span)c confirmando a renderização correta das especificações das rotas.
RNF-IMPL-01	Verificação de conformidade na importação de bibliotecas permitidas nas rotinas de desenvolvimento do backend.	Auditoria estática de código automatizada no ambiente de integração contínua impedindo pacotes externos não homologados.



Identificador RNF	Componente de Implementação Concreta	Estratégia de Verificação e Testes
RNF-IMPL-02	Inclusão física de scripts de Alpine.js e folhas de estilo locais do Tailwind CSS nas pastas estáticas da aplicação.	Testes de execução local completa isolando a máquina host de qualquer conectividade externa com a internet.

11. Conclusão

A especificação de requisitos não funcionais para o sistema Kanban Web estabelece uma arquitetura robusta e aderente às diretrizes de qualidade recomendadas pela disciplina de Engenharia de Software. A estruturação detalhada dos atributos de qualidade sob o padrão ISO/IEC 25010 assegura que as decisões técnicas - que englobam desde a escolha da stack tecnológica até a validação detalhada de segurança e o tratamento específico de eventos no frontend - sustentem as operações funcionais do sistema de forma resiliente.

A análise aprofundada de cenários críticos e complexos de concorrência, integrada a cálculos matemáticos formais de entropia, demonstra a viabilidade operacional e a robustez da solução, garantindo que o produto final seja seguro contra vulnerabilidades comuns. A matriz de rastreabilidade fornece à equipe um elo direto para validar a conformidade de cada requisito ao longo das etapas de desenvolvimento. Esse alinhamento metodológico resulta em um software estável, performático e de fácil manutenção, perfeitamente compatível com o ambiente dinâmico de desenvolvimento e colaboração de projetos de engenharia da universidade.



Referências citadas

1. Melhoria de um processo de engenharia de requisitos de software - UnB, https://bdm.unb.br/bitstream/10483/38579/1/2023_GiuliaLoboBarros_tcc.pdf
2. Requirements Sprint: Uma Abordagem Orientada a Requisitos de Software para Projetos Ágeis de Pequeno e Médio Portes - UnB, https://bdm.unb.br/bitstream/10483/30629/1/2021_KamillaCostaSouza_tcc.pdf
3. Documento de arquitetura - Alohomora, https://alohomora-team.github.io/2019.2-AlohomoraPage/projeto/doc_arquitetura/
4. Sprint 2 - Documentação - Requisitos Funcionais e Não Funcionais · Issue #17 · fga-eps-mds/2021.2-INDICAA - GitHub, <https://github.com/fga-eps-mds/Projeto01/issues/17>
5. Documento de Visão - UnBrake, <https://fga-eps-mds.github.io/2019.1-unbrake/documentos/documento-de-visao>
6. Building REST APIs using FastAPI, SQLAlchemy & Uvicorn | by Suman Das | Medium, <https://dassum.medium.com/building-rest-apis-using-fastapi-sqlalchemy-uvicorn-8a163ccf3aa1>
7. The FAST Stack - FastAPI + Astro + SQLite + Tailwind : r/webdev - Reddit, https://www.reddit.com/r/webdev/comments/1ou0x4s/the_fast_stack_fastapi_astro_sqlite_tailwind
8. NFR - Requisitos - Yellow, <https://yellow.netlify.app/analise/verificacao/nfr/>
9. FastAPI + HTMX: The No-Build Full-Stack - Blake Crosley, <https://blakecrosley.com/guides/fastapi-htmx>
10. Building a CRUD FastAPI app with SQLAlchemy - Mattermost, <https://mattermost.com/blog/building-a-crud-fastapi-app-with-sqlalchemy>
11. How to Use Async Database Connections in FastAPI - OneUptime, <https://oneuptime.com/blog/post/2026-02-02-fastapi-async-database/view>
12. Full-stack FastAPI with HTMX and Tailwind - FastAPI Setup | TestDriven.io, <https://testdriven.io/courses/fastapi-htmx/fastapi-setup>
13. SQL (Relational) Databases - FastAPI, <https://fastapi.xiniushu.com/az/tutorial/sql-databases>
14. Python Boilerplate - How to use Tailwind CSS and SQLAlchemy in your FastAPI app | Ezz, <https://ezzeddinabdullah.com/post/fastapi-tailwind-sqlalchemy>
15. dhtmlxScheduler with FastAPI | DHTMLX Scheduler Docs, <https://docs.dhtmlx.com/scheduler/de/integrations/python/howtostart-fastapi>
16. Roadmap dos papéis por sprint - UnBrake, <https://fga-eps-mds.github.io/2019.1-unbrake/roadmaps/roadmap-papeis>
17. Validation Rules Documentation - GitLab, https://code.swecha.org/corpus/corpus-server-app/-/blob/main/docs/VALIDATION_RULES.md
18. Financial KPI Extraction using Large Language Models - Lund University Publications, <https://lup.lub.lu.se/student-papers/record/9221154/file/9221160.pdf>
19. From Zero to Virtue: My Journey Building a Character-Level LLM, API, and Streaming Web App - Medium, <https://medium.com/@austinantony06/from-zero-to-virtue-my-journey-building-a-character-level-llm-api-and-streaming-web-app-5f415fc735ff>
20. Documento de arquitetura - Dr. Down, https://fga-eps-mds.github.io/2018.1-Dr-Down/mds/ARCHITECTURE_DOCUMENT
21. Building a Modern Kanban Board with Vanilla JavaScript: A Complete Guide to Drag-and-Drop Task Management - Medium, <https://medium.com/@francesco-saviano/building-a-modern-kanban-board-with-vanilla-javascript-a-complete-guide-to-drag-and-drop-task-4f1d1b27387f>
22. Sort - Alpine.js, <https://alpinejs.dev/plugins/sort>
23. Kanban board with drag and drop - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API/Kanban_board
24. HTML5 dragleave fired when hovering a child element - Stack Overflow, <https://stackoverflow.com/questions/7110353/html5-dragleave-fired-when-hovering-a-child-element>
25. FastAPI with Async SQLAlchemy, SQLModel, and Alembic | TestDriven.io, <https://testdriven.io/blog/fastapi-sqlmodel>